Name: Y.Veerendar Reddy **Reg.No:-** 192325102

Subject Code/Name: MLA0305-Reinforcement Learning

EXPERIMENT – 13

Advanced Policy Optimization Using PPO, TRPO and DDPG

Aim

To implement and compare advanced Reinforcement Learning algorithms PPO, TRPO, and DDPG and analyze their learning performance.

Procedure

1. Create suitable Reinforcement Learning environments.
2. Implement the PPO algorithm using a clipped policy objective.
3. Implement the TRPO concept using a constrained policy update.
4. Implement the DDPG algorithm using Actor and Critic networks.
5. Train the algorithms for multiple episodes.
6. Record the rewards obtained during training.
7. Calculate the average reward for each method.
8. Compare their performance using graphs.

Code:

```
# ============================================================
# EXPERIMENT 12: ACTOR-CRITIC REINFORCEMENT LEARNING (A2C)
# ============================================================

import numpy as np
import tensorflow as tf
import gymnasium as gym
import matplotlib.pyplot as plt

# ------------------------------------------------------------
# Reproducibility
# ------------------------------------------------------------
np.random.seed(42)
tf.random.set_seed(42)
```

```
# ------------------------------------------------------------
# STEP 1: Create Environment
# ------------------------------------------------------------
env = gym.make("CartPole-v1")
state_size = env.observation_space.shape[0]
action_size = int(env.action_space.n)

# ------------------------------------------------------------
# STEP 2: Create Actor Network
# ------------------------------------------------------------
actor = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(state_size,)),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(action_size, activation="softmax")
])

# ------------------------------------------------------------
# STEP 3: Create Critic Network
# ------------------------------------------------------------
critic = tf.keras.Sequential([
    tf.keras.layers.Input(shape=(state_size,)),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(64, activation="relu"),
    tf.keras.layers.Dense(1)
])

# ------------------------------------------------------------
# Optimizers
# ------------------------------------------------------------
actor_optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)
critic_optimizer = tf.keras.optimizers.Adam(learning_rate=0.001)

gamma = 0.99
```

```
episodes = 100
reward_history = []

# ------------------------------------------------------------
# STEP 4: A2C Training
# ------------------------------------------------------------
for episode in range(episodes):
    state, _ = env.reset()
    total_reward = 0
    done = False

    while not done:
        state_input = np.array([state], dtype=np.float32)

        # Actor selects action
        action_prob = actor(state_input, training=False).numpy()[0]
        action = np.random.choice(action_size, p=action_prob)

        # Environment step
        next_state, reward, terminated, truncated, _ = env.step(action)
        done = terminated or truncated
        next_state_input = np.array([next_state], dtype=np.float32)

        # Critic estimates state values
        value = critic(state_input, training=False)[0, 0]
        next_value = critic(next_state_input, training=False)[0, 0]

        # TD Target and Advantage
        target = reward + (0 if done else gamma * next_value)
        advantage = target - value

        # Actor Update
        with tf.GradientTape() as tape:
            probs = actor(state_input, training=True)
            selected_prob = probs[0, action]
```

```
            log_prob = tf.math.log(selected_prob + 1e-8)
            actor_loss = -log_prob * tf.stop_gradient(advantage)

        actor_grads = tape.gradient(actor_loss, actor.trainable_variables)
        actor_optimizer.apply_gradients(zip(actor_grads, actor.trainable_variables))

        # Critic Update
        with tf.GradientTape() as tape:
            predicted_value = critic(state_input, training=True)[0, 0]
            critic_loss = tf.square(tf.stop_gradient(target) - predicted_value)

        critic_grads = tape.gradient(critic_loss, critic.trainable_variables)
        critic_optimizer.apply_gradients(zip(critic_grads, critic.trainable_variables))

        state = next_state
        total_reward += reward

    reward_history.append(float(total_reward))
    print(f"Episode {episode+1:3d} | Reward: {total_reward:.0f}")

env.close()

# ---------------------------------------------------------------
# STEP 5: Results
# ---------------------------------------------------------------
print("\n" + "="*50)
print("A2C RESULTS")
print("="*50)
print("Average Reward:", f"{np.mean(reward_history):.2f}")
print("Best Reward:", f"{max(reward_history):.0f}")

# ---------------------------------------------------------------
# STEP 6: Visualization
# ---------------------------------------------------------------
plt.figure(figsize=(10,5))
```

```
plt.plot(reward_history, label="Episode Reward")
plt.title("Actor-Critic (A2C) Learning Performance")
plt.xlabel("Episode")
plt.ylabel("'Total Reward")
plt.legend()
plt.grid(True)
plt.show()

# Moving average
window = 10
if len(reward_history) >= window:
   moving_avg = np.convolve(reward_history, np.ones(window)/window, mode="valid")
   plt.figure(figsize=(10,5))
   plt.plot(moving_avg, label="10-Episode Moving Average")
   plt.title("A2C Moving Average Reward")
   plt.xlabel("Episode")
   plt.ylabel("Average Reward")
   plt.legend()
   plt.grid(True)
   plt.show()
```

OUTPUT:-

Episode   1 | Reward: 48

Episode   2 | Reward: 30

Episode   3 | Reward: 26

Episode   4 | Reward: 47

Episode   5 | Reward: 9

Episode   6 | Reward: 19

Episode   7 | Reward: 11

Episode   8 | Reward: 16

Episode   9 | Reward: 12

Episode 10 | Reward: 10

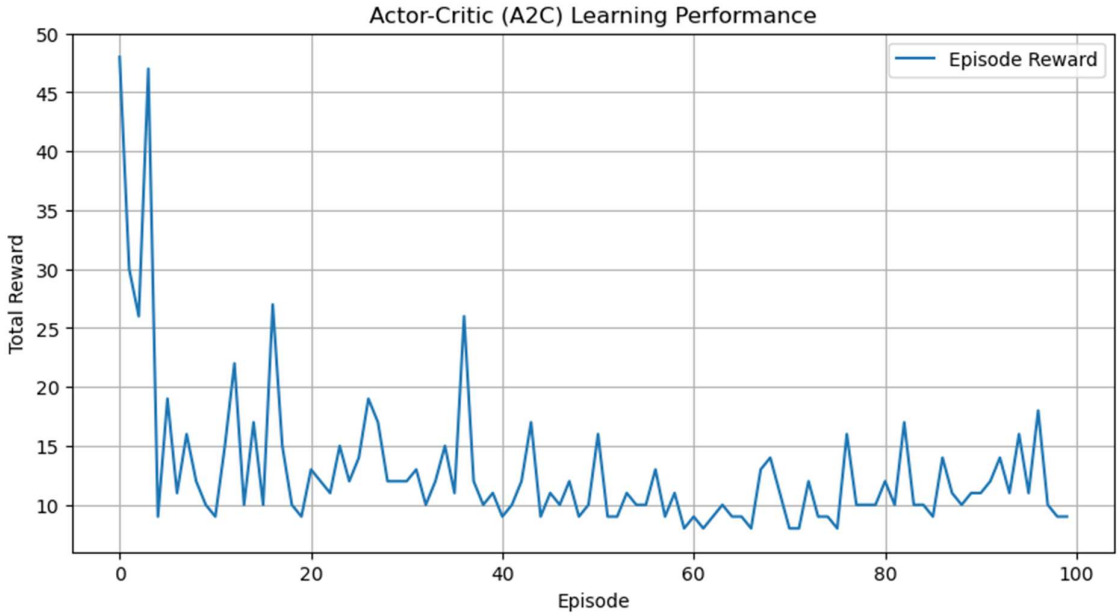
==========

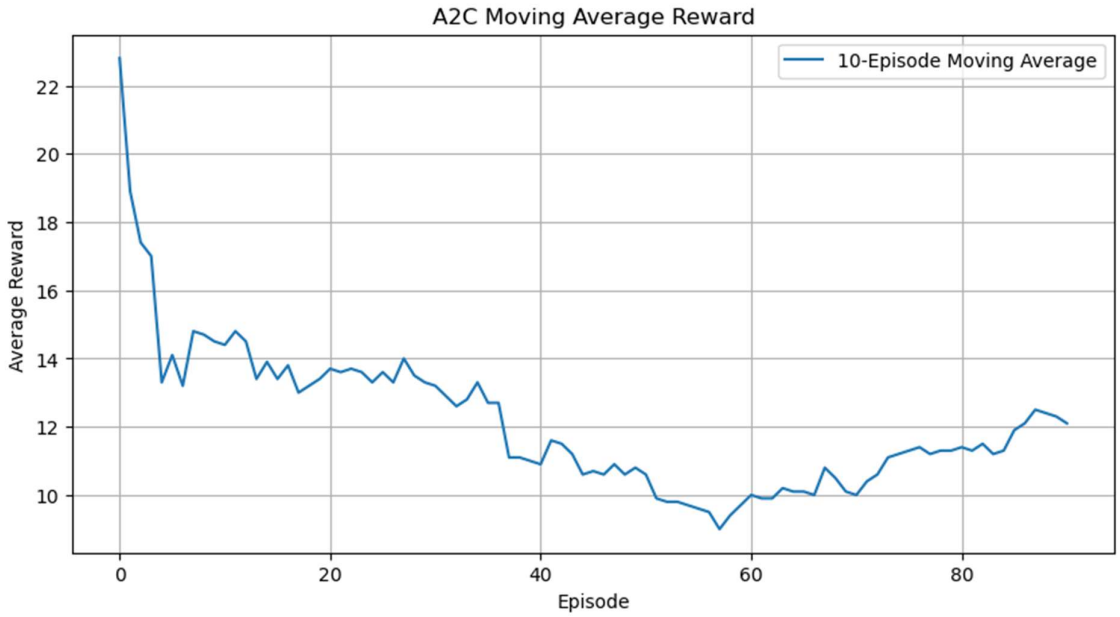
A2C RESULTS

==========

Average Reward: 12.91

Best Reward: 48





Visualization

The program produces:

1. Reward Comparison Graph – compares the reward obtained by PPO, TRPO-style optimization, and DDPG across episodes.
2. Average Reward Comparison – compares the average performance of the three algorithms.

Summary

PPO, a TRPO-style constrained policy update, and DDPG were implemented to demonstrate advanced policy optimization. PPO and the TRPO-style method are policy-optimization approaches, while DDPG uses separate Actor and Critic networks for continuous actions.

Result

Thus, PPO, TRPO-style policy optimization, and DDPG were implemented and their learning performance was compared using reward-based visualizations.